

Analytical Skills for Business (WS 2025/26)

Business Administration (M. A.)

© Benjamin Gross

24.10.2025 10:58

This document holds the course material for the Analytical Skills for Business course in the Master of Business Administration program. It discusses version control systems such as Git and GitHub for efficient team collaboration, offers an overview of no-code and low-code tools for data analytics including Tableau, Power BI, QlikView, makeML, PyCaret, RapidMiner, and KNIME, and introduces key programming languages such as R, Python, and SQL alongside essential programming concepts like syntax, libraries, variables, functions, objects, conditions, and loops. In addition, it covers working with modern development environments, including Unix-like systems, containers, APIs, Jupyter, and RStudio, and sets expectations for project submissions and evaluation.

Table of contents

1	Introduction	3
1.1	Implementing version control systems	3
1.1.1	Core Concepts	3
1.1.2	Git: Distributed Version Control	5
1.1.3	GitHub: Cloud-Based Collaboration Platform	5
1.1.4	Comparison of Git and GitHub	7
1.1.5	Business Analytics Applications	8
1.1.6	A lot of documentation available	8
1.2	Overview on Programming languages	9
1.3	Elements of programming languages	9
1.4	Development environments	9
1.5	Overview on no-code and low-code tools for data analytics	10
2	Descriptive statistics	13
2.0.1	1. Central Tendency	13
2.0.2	2. Variability	14
2.0.3	3. Distribution Shape	14
2.0.4	4. Frequencies and Percentiles	15
2.0.5	5. Numerical Methods	16
2.0.6	6. Graphical Methods	17
2.1	Measures of centrality, dispersion, and concentration	19
2.2	Descriptive analytics	19
2.3	Techniques	19
2.4	Visualizing and exploration	19

2.5	Handling messy data	19
2.6	Association	19
2.7	Implementing applications	19
3	Inferential statistics	20
3.1	Basic concepts of statistical inference	20
3.2	Quantification of probability through random variables	20
3.3	Hypothesis testing	20
3.3.1	Core Concepts	20
3.3.2	Key Components	20
3.3.3	Types of Errors	20
3.3.4	Reference Materials	21
3.4	Confidence intervals, p-values, and statistical tests	21
3.5	Inferential statistics	21
4	Predictive analytics	22
4.1	Data mining techniques	22
4.2	Regression analysis	22
4.3	Forecasting in predicting future business outcomes	22
5	Literature	23
5.1	Essential Readings	23
5.2	Further Readings	23
6	Example Exam	24

1 Introduction

Computer science is the study of computers and computation, spanning theoretical and algorithmic foundations, the design of hardware and software, and practical uses of computing to process information. It encompasses core areas such as

- algorithms and data structures
- computer architecture
- programming languages and software engineering
- databases and information systems
- networking and communications
- graphics and visualization
- human-computer interaction
- intelligent systems.

The field draws on mathematics and engineering—using concepts like

- binary representation
- Boolean logic
- complexity analysis

to reason about what can be computed and how efficiently.

Emerging in the 1960s as a distinct discipline, computer science now sits alongside computer engineering, information systems, information technology, and software engineering within the broader computing family. Its reach is inherently interdisciplinary, intersecting with domains from the natural sciences to business and the social sciences. Beyond technical advances, the discipline engages with societal and professional issues, including

- reliability
- security
- privacy
- intellectual property

in a networked world (Britannica, 2025).

1.1 Implementing version control systems

Version control systems are essential tools for managing code, tracking changes, and facilitating collaborative development in modern development projects (Çetinkaya-Rundel & Hardin, 2021). These systems enable teams to work efficiently on shared codebases while maintaining a complete history of all modifications, ensuring reproducibility and accountability in data analysis workflows.

1.1.1 Core Concepts

Version control systems provide systematic approaches to managing changes in documents, programs, and other collections of information:

- **Repository:** A central storage location containing all project files and their complete revision history

- **Commit:** A snapshot of the project at a specific point in time, representing a set of changes
- **Branch:** An independent line of development allowing parallel work on different features
- **Merge:** The process of integrating changes from different branches back together

Figure 1: **Source:** <https://uidaholib.github.io/get-git/1why.html>



1.1.2 Git: Distributed Version Control

Git is a distributed version control system that tracks changes in files and coordinates work among multiple contributors. It was created by **Linus Torvalds** (creator of Linux) in 2005 and has since become the de facto standard for version control in software development. Key characteristics include:

Local Repository: Each user maintains a complete copy of the project history, enabling offline work and faster operations.

Staging Area: An intermediate area where changes are prepared before being committed to the repository.

Branching and Merging: Lightweight branching allows for experimental development without affecting the main codebase. Merging integrates changes from different branches. In Open Source projects often Pull Requests are used to propose and discuss changes before merging. In the corporate world often Merge Requests are used. There is a difference between merging and rebasing. Merging creates a new commit that combines the histories of two branches, while rebasing rewrites the commit history to create a linear sequence of changes as you see on the figure below.

Figure 2: **Source:** <https://i0.wp.com/digitalvarys.com/wp-content/uploads/2020/03/Git-Merge-and-Rebase.png?resize=1536%2C843&ssl=1>



Distributed Workflow: No single point of failure, as every user has a complete backup of the project.

1.1.3 GitHub: Cloud-Based Collaboration Platform

GitHub is a web-based hosting service for Git repositories that adds collaboration features and project management tools:

- **Remote Repositories:** Centralized storage accessible from anywhere with internet connectivity.
- **Pull Requests:** Structured code review process for integrating changes.
- **Issue Tracking:** Built-in project management for tracking bugs and feature requests.
- **Actions and CI/CD:** Automated workflows for testing and deployment.
- **Documentation:** Integrated wiki and README support for project documentation.

The combination of Git and GitHub creates a powerful ecosystem for collaborative analytics projects, ensuring code quality, facilitating peer review, and maintaining comprehensive project documentation (G. GeeksforGeeks, 2024).

See also [Collaborating with Git and GitHub](#) by Prof. Dr. Huber about using git and Github for collaboration.

For students Github offers a [free educational](#) plan with additional features! Included you will find access to Github Copilot, a AI based code completion tools like. These are powerful gadgets to support your coding activities, but not a replacement for learning programming and coding by yourself as you can see on this image:

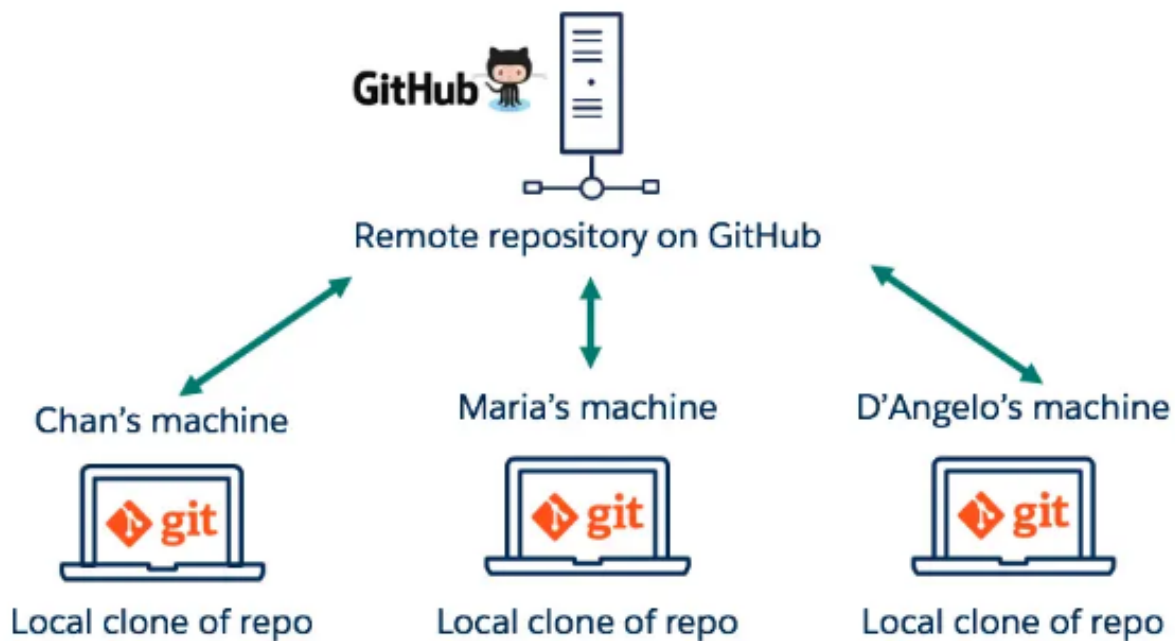
Figure 3: **Source:** *LinkedIn*

	Vibe Coding	VS Vibe Engineering
 Goal	Make it run now	Solve real problems at scale. Monetize it. Make it reliable and safe.
 Prompting	Casual: "make X"	Structured: roles, security, components, data rules, background jobs
 Tech skills	Not needed	System design, APIs, tools, and services, e.g., Netlify, RLS, XSS, DevTools, MCP, and agents
 Coding skills	Not needed	Just enough to review & question AI's work: logical expressions, functions, etc.
 Risks	Hidden bugs, data leaks, fragile code	More effort upfront, but fewer crises later
 Feels like	Using cool tech, copy-pasting whatever AI spits out	Designing flows, managing the context, testing, defining KPIs and alerts, monitoring logs
 Best for	Demos, experiments, prototypes	Real products, sensitive data, customer use
 Career impact	Might just delay being replaced by AI	Helps you capture new opportunities and become the one who controls AI
productcompass.pm		

1.1.4 Comparison of Git and GitHub

On this image you can see the integration of Git in GitHub:

Figure 4: **Source:** https://marce10.github.io/ciencia_reproducible/intro_a_git_y_github.html



1.1.5 Business Analytics Applications

In business analytics contexts, version control systems provide:

- **Reproducible Analysis:** Complete tracking of analytical scripts and data processing steps
- **Collaborative Research:** Multiple analysts can work simultaneously on different aspects of projects
- **Model Versioning:** Systematic management of machine learning models and their evolution
- **Data Governance:** Audit trails for compliance and regulatory requirements
- **Backup and Recovery:** Protection against data loss and accidental modifications

1.1.6 A lot of documentation available

For more coverage of version control concepts, implementation strategies, and best practices, see:

- **Theoretical Foundation:** [Introduction to Modern Statistics](#) provides context on reproducible research practices and collaborative analytics (Çetinkaya-Rundel & Hardin, 2021)
- **Git Resources:** [Git Cheat Sheet](#) offers quick reference for common Git commands
- **GitHub Documentation:** [GitHub Manual](#) contains detailed guidance on platform features
- **Online Resources:**
 - [GeeksforGeeks Git vs GitHub Guide](#) provides practical comparisons and use cases (GeeksforGeeks, 2024)
 - [Official GitHub Documentation](#) offers authoritative guidance on getting started (GitHub, 2024)

Understanding version control systems is fundamental for modern business analytics, enabling collaborative development, ensuring reproducibility, and maintaining professional standards in data science projects.

1.2 Overview on Programming languages

- **R:** R is a programming language and free software environment used for statistical computing and graphics supported by the R Foundation for Statistical Computing. It is widely used among statisticians and data miners for developing statistical software and data analysis.
- **Python:** Python is a versatile programming language widely used in data science and analytics. It has a rich ecosystem of libraries such as Pandas, NumPy, and Matplotlib that facilitate data manipulation, analysis, and visualization. It can also be used to develop software as it is a general-purpose programming language which utilizes an object-oriented programming paradigm.
- **SQL:** SQL (Structured Query Language) is the standard language for managing and querying relational databases. It is essential for data extraction, transformation, and loading (ETL) or extraction, loading and transformation (ELT) processes in analytics workflows. Modern databases like PostgreSQL, MySQL, and SQLite use SQL for data manipulation and retrieval and just have slightly different dialects.

1.3 Elements of programming languages

- **Syntax:** the set of rules that defines the combinations of symbols that are considered to be correctly structured programs in that language.
- **Libraries:** collections of pre-written code that users can call upon to save time and effort.
- **Variables:** named storage locations in a program that hold values.
- **Functions:** reusable blocks of code that perform a specific task.
- **Objects:** instances of classes that encapsulate data and behavior.
- **Conditions:** statements that control the flow of execution based on certain criteria.
- **Loops:** constructs that repeat a block of code multiple times.

1.4 Development environments

- **Unix-like systems:** Most popular Unix like system is [Linux](#). It is understood as a family of open source Unix-like operating systems based on the Linux kernel and mostly used in form of distributions such as [Ubuntu](#), [Debian](#), [Fedora](#), [CentOS](#) and [Alpine Linux](#). The latter is known for its simplicity and efficiency and is often used as the base image for Docker containers (container host os).
- **Containers:** [Docker](#) is a production ready containerization service. On [hub.docker.com](#) you can store images that could be pulled. It acts as the main public registry for Docker images. Containers are a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another. A Docker container image is a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries and settings.
- **APIs:** (Application Programming Interface): An API is a set of definitions and protocols for building and integrating application software. It allows different software systems to communicate with each other. APIs are used to enable the integration of different systems,

allowing them to share data and functionality. Examples include RESTful APIs, SOAP APIs, and GraphQL APIs.

- **Jupyter:** [Jupyter](#) is an open-source web application that allows you to create and share documents that contain live code, equations, visualizations, and narrative text. It supports various programming languages, including Python, R, and Julia. Jupyter is widely used for data analysis, machine learning, and scientific computing.
- **IDEs:** (Integrated Development Environments): IDEs are software applications that provide comprehensive facilities to computer programmers for software development. They typically include a code editor, a debugger, and build automation tools. Examples of popular IDEs include:
 - **RStudio:** [RStudio](#) is an integrated development environment (IDE) for R, a programming language for statistical computing and graphics. RStudio provides a user-friendly interface for writing and debugging R code, as well as tools for data visualization and reporting.
 1. Install R from [CRAN](#)
 2. Install RStudio from [RStudio](#)
 3. Install git (if not already installed) from [git-scm](#)
 4. Follow the lecture to clone the Course GitHub repository and open the project in RStudio <https://github.com/DrBenjamin/Analytical-Skills-for-Business>
 - **Visual Studio Code (VS Code):** [VS Code](#) is a free source-code editor made by Microsoft for Windows, Linux and macOS. It includes support for debugging, embedded Git control, syntax highlighting, intelligent code completion, snippets, and code refactoring. It is highly customizable, allowing users to change the theme, keyboard shortcuts, preferences, and install extensions that add additional functionality.

1.5 Overview on no-code and low-code tools for data analytics

- [n8n](#)
 - Follow these short introductions to n8n workflows:
 - * [Data downloading](#) (import it with the context menu on the top right of the workflow page (...) - Import from File... or Import from URL...):
 1. Create an account on [n8n.cloud](#)
 2. Create a new workflow
 3. Add a new node and select “HTTP Request”
 4. Configure the node to make a GET request to <https://minio.seriousbenentertainment.org/202025.csv>
 5. Add a new node and select “Write Binary File”
 6. Configure the node to write the data to your local disk
 - * [Mediapipe](#) (import it with the context menu on the top right of the workflow page (...) - Import from File... or Import from URL...):
 1. Create a new workflow
 2. Add a new node and select “HTTP Request” - configure the node to make a GET request to <https://minio.seriousbenentertainment.org:9000/data/input.mp4>
 3. Add a new node Extract from File
 4. Add a new node Build JSON
 5. Add a new node and select “MediaPipe”
 6. Add a new node HTTP Request - configure the node to make a POST request to <http://212.227.102.172:8000/mediapipe>

7. Add a new node Github - configure the node to commit the results to the [Github repository](#)

- **Streamlit on Snowflake**

- Follow these short introductions to Snowflake:
 1. Create a free trial account on [Snowflake](#)
 2. Choose the data tutorial and follow the steps.

- **QGIS**

- Follow these short introductions to QGIS:
 1. Download and install QGIS from [here](#)
 2. Download the sample data from:
 - [here](#) and unzip them into one folder - it needs to contain all these files and sub-folders: Day2Data Day4Data Malawi-healthsites mw_districts mw_shp Malawi.qgz
 3. Open QGIS and open the project file Malawi.qgs
 4. Open [Department of Forestry](#) website and download some or all of the zip files.
 5. Add the layers via the menu **Layer -> Add layer -> Add Vector Layer** and select the shapefiles from the unzipped folders.

- **Tableau**

- Follow these short introductions to Tableau:
 1. Download and install [Tableau Desktop](#) which is free for students or use [Tableau Public](#) online.
 2. Download the sample data from [here](#) and open it in Tableau.

- **Power BI**

- Follow these short introductions to Power BI:
 1. We are using Power BI within MS Teams (it would also be available [here](#) to use as Power BI Desktop Application).
 2. Open an existing Power BI project and make yourself familiar with the interface.

- **KNIME**

- Follow these short introductions to KNIME:
 1. Download and install KNIME Analytics Platform from [here](#)
 2. Create a new KNIME workflow and add a CSV Reader node to read data from a CSV file. Configure the node to use Custom/KNIME URL and add <https://raw.githubusercontent.com/datasciencedojo/datasets/refs/heads/master/titanic.csv> in the URL field. Configure the node to properly read column headers and data types.
 3. Explore the Data - Add a “Statistics” node to understand the data distribution
 - Use the “Missing Value” node to identify missing values in columns like Age, Cabin, and Embarked
 4. Handle Missing Values
 - * For the Age column: use “Missing Value” node with mean or median imputation

- * For the Cabin column: create a new binary feature indicating whether cabin information is available
 - * For the Embarked column: use mode imputation or remove the few rows with missing values
5. Data Transformation
- * Use “Column Filter” to remove unnecessary columns (e.g., PassengerId, Name, Ticket)
 - * Apply “String Manipulation” for categorical encoding
 - * Use “Normalizer” node for numerical features if needed
6. Export Cleaned Data
- * Use “CSV Writer” node to save the cleaned dataset
 - * Verify the output and check data quality

2 Descriptive statistics

Descriptive statistics summarizes and presents the main features of a dataset so you can understand what the data look like before modeling or inference. It organizes raw values into clear numerical summaries and visuals, without making probabilistic claims about a wider population. In analytics projects, this first pass helps you validate data quality, spot outliers, and communicate patterns to stakeholders.

See [Types of Data](#) and [Types of Variables](#) to build an data wrangling foundation for descriptive statistics.

What we typically summarize:

2.0.1 1. Central Tendency

Central tendency measures identify the typical or central value in a dataset.

- The **mean** (arithmetic average) is sensitive to extreme values, making it suitable for symmetric distributions. - The **median** (middle value when sorted) is robust to outliers and preferred for skewed data. The **mode** (most frequent value) is useful for categorical data and can identify the most common category or value. In business analytics, choosing the appropriate measure depends on data distribution and the presence of outliers.
- The **median** (middle value when sorted) is robust to outliers and preferred for skewed data.
- The **mode** (most frequent value) is useful for categorical data and can identify the most common category or value. In business analytics, choosing the appropriate measure depends on data distribution and the presence of outliers.

```
# Loading necessary libraries
library(tidyverse)

# Creating a sample dataset
sample_data <- tibble(
  values = c(1, 2, 2, 3, 4, 5, 100) # Example data with an outlier
)

# Calculating central tendency measures
central_tendency <- sample_data %>%
  summarise(
    mean = mean(values),
    median = median(values),
    mode = as.numeric(names(sort(table(values), decreasing = TRUE)[1]))
  )

# Printing the results
print(central_tendency)
```



```
# A tibble: 1 x 3
  mean median mode
<dbl> <dbl> <dbl>
1  16.7      3     2
```

2.0.2 2. Variability

Variability measures quantify data spread around the center.

- The **range** (maximum minus minimum) provides a simple spread indicator but is sensitive to outliers.
- The **Variance** measures average squared deviations from the mean, while
- The **standard deviation** (square root of variance) expresses spread in original units, facilitating interpretation.
- The **interquartile range (IQR)** (difference between 75th and 25th percentiles) is robust to outliers and describes middle 50% spread. In business contexts, variability indicates risk, consistency, or process stability.

```
# Calculating variability measures
variability <- sample_data %>%
  summarise(
    range = max(values) - min(values),
    variance = var(values),
    sd = sd(values),
    iqr = IQR(values)
  )

# Printing the results
print(variability)
```

```
# A tibble: 1 x 4
  range variance    sd  iqr
<dbl>    <dbl> <dbl> <dbl>
1     99    1351.  36.8  2.5
```

2.0.3 3. Distribution Shape

Distribution shape characteristics reveal asymmetry and tail behavior.

- The **Skewness** measures distribution asymmetry: positive skewness indicates a right tail (mean > median), negative skewness indicates a left tail (mean < median), and zero indicates symmetry.
- The **Kurtosis** measures tail heaviness and peakedness: high kurtosis indicates heavy tails with more outliers, low kurtosis indicates light tails.

These measures help identify data transformations needed and potential outliers in business analytics.

```
# Loading moments package for skewness and kurtosis
library(tidyverse)
library(moments)

# Calculating distribution shape measures
distribution_shape <- sample_data %>%
  summarise(
    skewness = skewness(values),
```

```

    kurtosis = kurtosis(values)
  )

# Printing the results
print(distribution_shape)

```

```

# A tibble: 1 x 2
  skewness kurtosis
    <dbl>    <dbl>
1     2.04     5.15

```

2.0.4 4. Frequencies and Percentiles

Frequency analysis counts occurrences and calculates proportions, revealing data distribution patterns.

- **Percentiles** (quantiles) divide data into equal parts: quartiles (25%, 50%, 75%), deciles (10% intervals), or any custom percentile.

These measures identify data position and help detect outliers. In business, percentiles benchmark performance (e.g., top 10% customers) and support decision thresholds.

```

# Loading moments package for skewness and kurtosis
library(tidyverse)

# Calculating frequencies and percentiles
frequencies <- sample_data %>%
  count(values) %>%
  mutate(proportion = n / sum(n))

# Printing frequency table
print(frequencies)

```

```

# A tibble: 6 x 3
  values      n proportion
    <dbl> <int>    <dbl>
1      1      1    0.143
2      2      2    0.286
3      3      1    0.143
4      4      1    0.143
5      5      1    0.143
6     100      1    0.143

```

```

# Calculating various percentiles
percentiles <- sample_data %>%
  summarise(
    q25 = quantile(values, 0.25),
    q50 = quantile(values, 0.50), # median
    q75 = quantile(values, 0.75),

```

```

    q90 = quantile(values, 0.90),
    q95 = quantile(values, 0.95)
  )

# Printing percentiles
print(percentiles)

```

```

# A tibble: 1 x 5
  q25    q50    q75    q90    q95
<dbl> <dbl> <dbl> <dbl> <dbl>
1     2     3    4.5  43.0  71.5

```

What are common methods to summarize:

2.0.5 5. Numerical Methods

Numerical methods systematically compute summary statistics and organize data into interpretable structures.

- The **Summary metrics** condense datasets into key statistics (mean, median, standard deviation, quartiles), enabling quick assessment.
- The **Frequency tables** tabulate value counts or binned ranges, revealing distribution patterns. These methods form the foundation for exploratory data analysis and inform subsequent statistical modeling in business analytics.

```

# Creating a more comprehensive summary
summary_metrics <- sample_data %>%
  summarise(
    count = n(),
    min = min(values),
    q1 = quantile(values, 0.25),
    median = median(values),
    mean = mean(values),
    q3 = quantile(values, 0.75),
    max = max(values),
    sd = sd(values),
    variance = var(values)
  )

# Printing summary metrics
print(summary_metrics)

```

```

# A tibble: 1 x 9
  count    min    q1 median  mean    q3    max    sd variance
<int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>    <dbl>
1     7     1     2     3  16.7   4.5  100  36.8  1351.

```



```
# Creating a frequency table with binned ranges
binned_data <- sample_data %>%
  mutate(bin = cut(values, breaks = c(0, 2, 4, 6, Inf),
                        labels = c("0-2", "2-4", "4-6", ">6"))) %>%
  count(bin) %>%
  mutate(percentage = n / sum(n) * 100)

# Printing frequency table
print(binned_data)
```

```
# A tibble: 4 x 3
  bin      n percentage
<fct> <int>     <dbl>
1 0-2      3      42.9
2 2-4      2      28.6
3 4-6      1      14.3
4 >6       1      14.3
```

2.0.6 6. Graphical Methods

Graphical methods visualize data distributions and relationships, making patterns immediately recognizable.

- **Histograms** display continuous data frequency distributions across bins.
- **Bar and pie charts** compare categorical data frequencies or proportions.
- **Box plots** show median, quartiles, and outliers, facilitating distributional comparisons.
- **Scatter plots** reveal bivariate relationships and correlation patterns. In business analytics, visualizations communicate insights to non-technical stakeholders and guide data-driven decisions.

```
# Creating a larger dataset for better visualizations
set.seed(42)
large_data <- tibble(
  continuous_var = c(rnorm(100, mean = 50, sd = 10), rnorm(20, mean = 80, sd = 5)),
  category = sample(c("A", "B", "C", "D"), 120, replace = TRUE),
  x_var = rnorm(120, mean = 30, sd = 8),
  y_var = 2 * rnorm(120, mean = 30, sd = 8) + rnorm(120, mean = 0, sd = 5)
)

# Creating multiple plots
library(gridExtra)

# Histogram for continuous data
p1 <- ggplot(large_data, aes(x = continuous_var)) +
  geom_histogram(bins = 15, fill = "steelblue", color = "white") +
  labs(title = "Histogram (Continuous Data)", x = "Value", y = "Frequency") +
  theme_minimal()

# Bar chart for categorical data
p2 <- ggplot(large_data, aes(x = category)) +
```

```

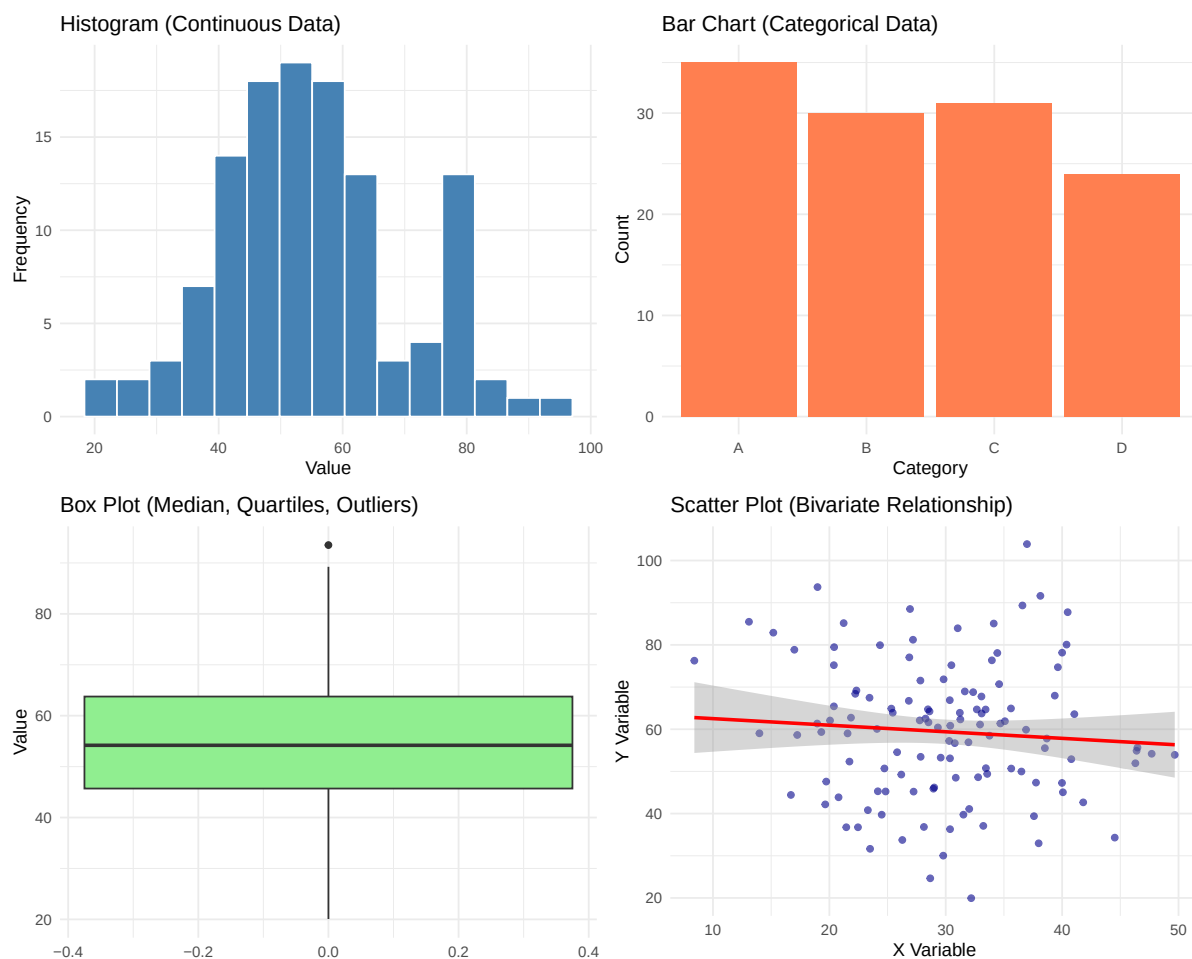
geom_bar(fill = "coral") +
labs(title = "Bar Chart (Categorical Data)", x = "Category", y = "Count") +
theme_minimal()

# Box plot
p3 <- ggplot(large_data, aes(y = continuous_var)) +
  geom_boxplot(fill = "lightgreen") +
  labs(title = "Box Plot (Median, Quartiles, Outliers)", y = "Value") +
  theme_minimal()

# Scatter plot for bivariate relationship
p4 <- ggplot(large_data, aes(x = x_var, y = y_var)) +
  geom_point(color = "darkblue", alpha = 0.6) +
  geom_smooth(method = "lm", color = "red", se = TRUE) +
  labs(title = "Scatter Plot (Bivariate Relationship)", x = "X Variable", y = "Y Variable") +
  theme_minimal()

# Arranging plots in a grid
grid.arrange(p1, p2, p3, p4, ncol = 2)

```



Business analytics context:

- For monthly revenue by region, the mean signals typical performance, the standard deviation shows volatility, a box plot quickly flags outliers, and a bar chart compares

regions. These summaries guide prioritization (e.g., regions with high variability may require deeper investigation) and set baselines for forecasting and experimentation.

- For an accessible overview of types, methods, and examples, see ResearchMethod.net.

2.1 Measures of centrality, dispersion, and concentration

2.2 Descriptive analytics

- univariate data
- bivariate data
- multivariate data

2.3 Techniques

- constructing
- interpreting
- evaluating of scores, rankings, metrics, and composite indicators.

2.4 Visualizing and exploration

- categorical
- numerical
- time series data

2.5 Handling messy data

2.6 Association

- measuring the association of variables, including correlation and regression

2.7 Implementing applications

in the programming language R for practical data analysis.

3 Inferential statistics

3.1 Basic concepts of statistical inference

3.2 Quantification of probability through random variables

3.3 Hypothesis testing

Hypothesis testing is a fundamental statistical method used to make inferences about population parameters based on sample data (Illowsky & Dean, 2018). It provides a systematic framework for evaluating claims about populations using sample evidence, enabling data-driven decision making in business contexts.

3.3.1 Core Concepts

A statistical hypothesis test involves formulating two competing hypotheses:

- **Null hypothesis (H_0)**: The status quo or default position, typically representing no effect or no difference
- **Alternative hypothesis (H_1 or H_a)**: The research hypothesis representing the effect or difference we seek to detect

The process involves calculating a test statistic from sample data and comparing it to a critical value or determining a p-value to make decisions about rejecting or failing to reject the null hypothesis (GeeksforGeeks, 2024).

3.3.2 Key Components

Test Statistics: Standardized measures that quantify how far sample data deviates from what would be expected under the null hypothesis.

Significance Level (α): The probability threshold for rejecting the null hypothesis, commonly set at 0.05 (5%).

P-value: The probability of observing test results at least as extreme as those obtained, assuming the null hypothesis is true.

Critical Region: The range of values for which the null hypothesis is rejected.

3.3.3 Types of Errors

- **Type I Error (α)**: Rejecting a true null hypothesis (false positive)
- **Type II Error (β)**: Failing to reject a false null hypothesis (false negative)
- **Statistical Power ($1 - \beta$)**: The probability of correctly rejecting a false null hypothesis

3.3.4 Reference Materials

For comprehensive coverage of hypothesis testing concepts, methodologies, and applications, consult:

- **Theoretical Foundation:** [Introductory Statistics](#) provides detailed explanations of hypothesis testing principles and procedures (Illowsky & Dean, 2018)
- **Visual Guide:** [Hypothesis Testing Overview](#) offers a visual representation of key concepts
- **Detailed Methodology:** [Hypothesis Testing Documentation](#) contains comprehensive methodological information
- **Online Resource:** Additional perspectives on hypothesis testing applications can be found in the [GeeksforGeeks guide](#) (GeeksforGeeks, 2024)

Understanding hypothesis testing is essential for making informed business decisions based on data analysis, forming the foundation for advanced statistical inference and predictive analytics in business contexts.

3.4 Confidence intervals, p-values, and statistical tests

xy

3.5 Inferential statistics

in the programming language R, translating theoretical knowledge into practical applications.

4 Predictive analytics

4.1 Data mining techniques

4.2 Regression analysis

4.3 Forecasting in predicting future business outcomes

5 Literature

All references for this course.

5.1 Essential Readings

Bruce, P. and A. Bruce (2020). *Practical Statistics for Data Scientists, 2nd Edition*. <https://learning.oreilly.com/library/view/practical-statistics-for/9781492072935/preface01.html>.

Çetinkaya-Rundel, M. and J. Hardin (2021). *Introduction to Modern Statistics*. <https://www.openintro.org/book/ims/>. https://github.com/DrBenjamin/Analytical-Skills-for-Business/blob/491a9a84dd0227aab44e0a6db7e6330830a05a6b/literature/Introduction_to_Modern_Statistics_2e.pdf?raw=true.

Stephenson, P. (2023). *Data Science Practice*. <https://datasciencepractice.study/>.

5.2 Further Readings

Békés, G. and G. Kézdi (2021). *Resources for Data Analysis for Business, Economics, and Policy*. Instructor resources: <https://www-cambridge-org.eux.idm.oclc.org/highereducation/books/data-analysis-for-business-economics-and-policy/D67A1B0B56176D6D6A92E27F3F82AA20/>. https://github.com/DrBenjamin/Analytical-Skills-for-Business/blob/c2ec1b2061c7dc36200977cfd58daf6020c1c774/literature/B%C3%A9k%C3%A9s_Data%20Analysis%20for%20Business%2C%20Economics%2C%20and%20Policy_2021_First%20Day%20of%20Class%20Slides.pdf?raw=true }

Britannica (2025). *Computer science | Definition, Types, & Facts | Britannica*. <https://www.britannica.com/science/computer-science>.

Dougherty, J. and I. Ilyankou (2025). *Hands-On Data Visualization*. <https://handsondataviz.org/>.

Evans, J. R. (2020). “Business Analytics”.

GeeksforGeeks (2024a). *Understanding Hypothesis Testing*. Online resource for software testing and statistical concepts. <https://www.geeksforgeeks.org/software-testing/understanding-hypothesis-testing/>.

GeeksforGeeks, G. (2024b). *Difference between Git and GitHub*. Comprehensive comparison of Git and GitHub for version control. <https://www.geeksforgeeks.org/git/difference-between-git-and-github/>.

GitHub (2024). *About GitHub and Git*. Official GitHub documentation on Git and GitHub fundamentals. <https://docs.github.com/en/get-started/start-your-journey/about-github-and-git>.

Illowsky, B. and S. L. Dean (2018). *Introductory Statistics*. OpenStax, Rice University, p. 905. ISBN: 1938168208. <https://github.com/DrBenjamin/Analytical-Skills-for-Business/blob/c2ec1b2061c7dc36200977cfd58daf6020c1c774/literature/Introductory%20Statistics.pdf?raw=true> }

Irizarry, R. A. (2024). “Advanced Data Science: Statistics and Prediction Algorithms Through Case Studies”.

<http://rafalab.dfci.harvard.edu/dsbook-part-2>.

Kumar, U. D. (2017). “Business Analytics: The Science of Data-Driven Decision Making”.

Pochiraju, B. and S. Seshadri, ed. (2019). *Essentials of Business Analytics*. Vol. 264. <https://link.springer.com/10.1007/978-3-319-68837-4>. Cham: Springer International Publishing. ISBN: 978-3-319-68836-7. DOI: 10.1007/978-3-319-68837-4. <https://github.com/DrBenjamin/Analytical-Skills-for-Business/blob/c2ec1b2061c7dc36200977cfd58daf6020c1c774/literature/Essentials%20of%20Business%20Analytics.pdf/?raw=true>.}

Vaughan, D. (2020). “Analytical Skills for AI and Data Science’”.

<https://learning.oreilly.com/library/view/analytical-skills-for/9781492060932/preface01.html#idm46388898852872>.

6 Example Exam

To prepare for the exam, please review the following example exam with solutions:

[Example Exam](#)